

Es. 2 induzione strutturale

$n \in E$, $e_1, e_2 \in E$ allora $e_1 \wedge e_2 \in E$, $e_1 \vee e_2 \in E$

$$\begin{cases} \kappa(n) = 1 \end{cases}$$

$$\begin{cases} \kappa(e_1 \wedge e_2) = \kappa(e_1 \vee e_2) = \kappa(e_1) + \kappa(e_2) \end{cases}$$

$$\begin{cases} o(n) = 0 \end{cases}$$

$$\begin{cases} o(e_1 \wedge e_2) = o(e_1 \vee e_2) = o(e_1) + o(e_2) + 1 \end{cases}$$

Dim.: $\forall e \in E$. $\kappa(e) = o(e) + 1$

n, e_1 e e_2 per essere usati nelle espressioni date sappiamo che devono essere booleani e quindi anche $e \in [\text{true}, \text{false}]$

Caso base:

$$\kappa(e) = o(e) + 1 \quad ; \quad 1 = 0 + 1 \quad \checkmark$$

Passo ind.:

$$e = e_1 \wedge e_2 \quad (\text{analogo per } e = e_1 \vee e_2)$$

$$\text{Hp. ind.: } \kappa(e_1) = o(e_1) + 1$$

$$\kappa(e_2) = o(e_2) + 1$$

$$\kappa(e_1) = o(e_1) + 1$$

Hp. ind.

$$\kappa(e) = \kappa(e_1 \wedge e_2) = \kappa(e_1) + \kappa(e_2) = o(e_1) + 1 + o(e_2) + 1$$

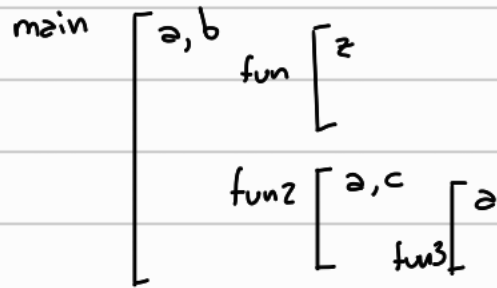
$$= o(e_1 \wedge e_2) + 1 = o(e) + 1 \quad \checkmark$$

Es. 3 scoping

```

{ int a=2; int b=3;
  void fun() { int z=a*b; }
  void fun2() {
    int a=4; int c=5;
    void fun3() {
      int a=c-b;
      fun(); }
    b=a+b;
    fun3(); }
  fun2(); }

```



profondità dichiarazioni

$$Sd(main) = 0$$

$$Sd(fun) = Sd(fun2) = 1$$

$$Sd(fun3) = 2$$

seq. chiamate

main → fun2 → fun3 → fun

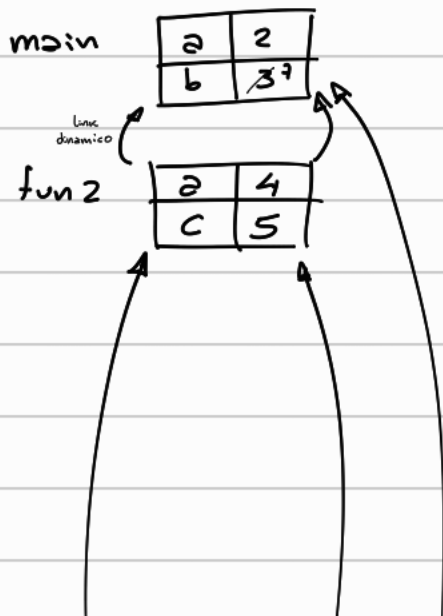
link statico $K = Sd(chiamante) - Sd(chiamato) + 1$

di volte da risolvere la catena statica

risolvere riferimenti: $N_a = Sd(uso) - Sd(dichiarazione)$

trovare RDA definizione

Scoping statico



$$\text{link statico } K_{fun2} = Sd(main) - Sd(fun2) + 1$$

$$= 0 - 1 + 1 = 0$$

$$CS(fun2) = main$$

esecuzione B: $b = a + b;$

$$- a \Rightarrow \text{locale} = 4$$

$$- b \Rightarrow N_b = Sd(fun2) - Sd(main) = 1 - 0 = 1$$

fun3

a	7-2
---	-----

fun

z	7-14
---	------

andiamo a prendere b dal main, b=3

link statico $K_{fun3} = Sd(fun2) - Sd(fun3) + 1$
 $= 1 - 2 + 1 = 0$

$CS(fun3) = fun2$

esecuzione fun3: $a = c - b$

- a => locale

- c => $N_c = Sd(fun3) - Sd(fun2) = 2 - 1 = 1$ | c=5

- b => $N_b = Sd(fun3) - Sd(main) = 2 - 0 = 2$ | b=7

preso da fun2

preso da main

link statico $K_{fun} = Sd(fun3) - Sd(fun) + 1$
 $= 2 - 1 + 1 = 2$

$CS(fun) = main$

esecuzione fun: $z = a * b$

- z => locale

- a => $N_a = Sd(fun) - Sd(main) = 1 - 0 = 1$ | a=2

- b => $N_b = Sd(fun) - Sd(main) = 1 - 0 = 1$ | b=7

presi da main

Dopo la chiamata tutte le chiamate terminano e lo stack deallocato.

Scoping dinamico

main

a
b
c
z

→ main, 2
 → main, 3

main → fun2

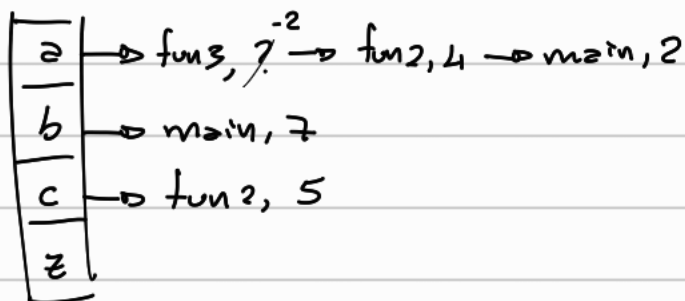
a
b
c
z

→ fun2, 4 → main, 2
 → main, 3
 → fun2, 5

esecuzione fun2

$b = a + b$

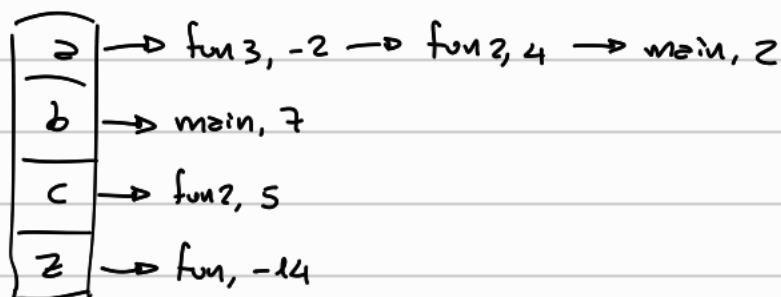
fun2 → fun3



esecuzione fun3

a = c - b

fun3 → fun



esecuzione fun

z = a * b

Es. 4 scoping

```
{ int i=0; int x=3;
```

(*)

```
void pippo() {
```

```
    int y;
```

(**)

```
    x = pluto();
```

```
    i = i + 2; }
```

```
while (i <= 2) { pippo(); }
```

→ stesso valore scoping statico

→ valori diversi scoping dinamico

(*) in questa zona di codice devo definire pluto perché non deve essere locale a pippo per poter rispettare le regole di scoping richieste

=> int pluto() { return x; }

(**) devo definire e inizializzare x

=> int x = i;

```
{ int i=0; int x=3;
```

```
int pluto() { return x; }
```

```
void pippo() {
```

```
    int y;
```

```
    int x=i;
```

```
    x = pluto();
```

```
    i=i+2; }
```

```
while (i<=2) { pippo(); } }
```

Scoping statico

La chiamata pluto restituisce la x globale che non viene mai modificata. Entrambe le chiamate restituiscono 3.

Scoping dinamico

La chiamata pluto restituisce la x interna al while che cambia con i. La prima volta $x=0$, la seconda $x=2$.

Es.5 ricorsione e ricorsione in coda

Def. ricorsione:

Metodo alternativo alla costruzione di procedure iterative.

È definita procedura ricorsiva se è definita in funzione di se stessa.

Def. ricorsione in coda:

Una procedura è detta in coda se la procedura chiamante restituisce il valore di tale procedura senza ulteriori computazioni.

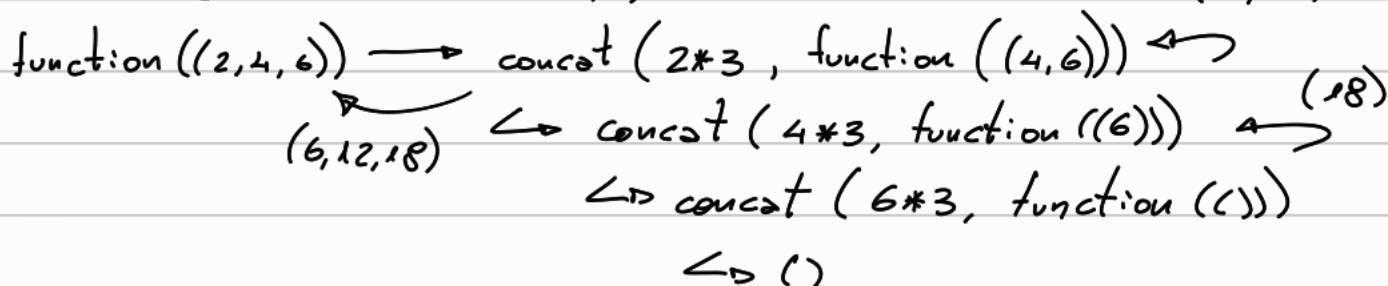
La procedura chiamante viene detta ricorsione in coda.

```
lista function (lista list) {
```

```
    if (length(list)=0) return ();
```

```
    else return concat(car(list)*3, function(cdr(list))); }
```

esecuzione con $list = (2, 4, 6)$



=> function restituisce la lista moltiplicando per 3 tutti gli elementi e mantiene l'ordine.

```
lista fun(lista list) {
  return fun-rc(list, ());
}
```

↗ valore base restituito nel caso list fosse vuota

```
lista fun-rc(lista list, lista res) {
  if (length(list) = 0) return res;
  else return fun-rc(cdr(list), concat(res, 3 * car(list)));
}
```

esecuzione con list = (2, 4, 6)

```
fun((2,4,6)) → fun-rc((2,4,6), ())
               ↙
               (6,12,18)
               ↗
               fun-rc((4,6), concat((), (3*2)))
                   ↙
                   (6,12,18)
                   ↗
                   fun-rc((6), concat((6), (3*4)))
                       ↙
                       (6,12,18)
                       ↗
                       fun-rc((), concat((6,12), (3*6)))
                           ↙
                           (6,12,18)
                           ↗
                           (6,12,18)
```

Es. 6 derivazione

$\rho = [x \mapsto 3, y \mapsto 4]$

$\sigma = [y \mapsto 2]$

$y := 3x + y$

$\Delta = [x \mapsto \text{int}, y \mapsto \text{intloc}]$

$$\frac{\frac{\Delta \vdash 3 : \text{int} \quad \Delta \vdash x : \text{int}}{\Delta \vdash 3x} \quad \Delta \vdash y : \text{int}}{\Delta \vdash 3x + y} \quad \Delta(y) = \text{intloc}$$

↓
se il risultato di $3x + y$ è int allora $y := 3x + y$ è ben formato

esecuzione

$$\frac{\rho \vdash \langle 3x+y, \sigma \rangle \longrightarrow^* \langle \kappa, \sigma \rangle}{\rho \vdash \langle y := 3x+y, \sigma \rangle \longrightarrow \langle y := \kappa, \sigma \rangle}$$

valutazione
di x

$$\frac{\rho \vdash \langle x, \sigma \rangle \longrightarrow \langle 3, \sigma \rangle}{\rho \vdash \langle 3x+y, \sigma \rangle \longrightarrow \langle 3 \cdot 3+y, \sigma \rangle} \quad \rho(x) = 3$$

valutazione
di y

$$\frac{\rho \vdash \langle y, \sigma \rangle \longrightarrow \langle 2, \sigma \rangle}{\rho \vdash \langle 3 \cdot 3+y, \sigma \rangle \longrightarrow \langle 3 \cdot 3+2, \sigma \rangle} \quad \rho(y) = l_y; \sigma(l_y) = 2$$

soluzione regola

$$\frac{\frac{\rho \vdash \langle 3 \cdot 3, \sigma \rangle \longrightarrow \langle 9, \sigma \rangle}{\rho \vdash \langle 3 \cdot 3+2, \sigma \rangle \longrightarrow \langle 11, \sigma \rangle}}{\rho \vdash \langle y := 3 \cdot 3+2, \sigma \rangle \longrightarrow \langle y := 11, \sigma \rangle}$$

$$\rho \vdash \langle y := 11, \sigma \rangle \longrightarrow \rho(y) = l_y \longrightarrow \sigma[l_y \mapsto 11]$$

Il valore nella locazione l_y (della variabile y) viene modificata in 11